

Deploying the Real-Time EEG System

Dr. Mohamed H. Zaky, Assistant Professor

Co-founder - WeBios Lab

Electronics and Communications Engineering Department
Arab Academy for Science, Technology and Maritime Transport



HERBioLab

Offline vs. online: What changes

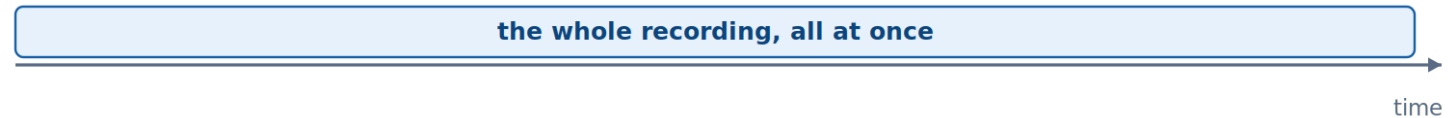
Offline you hold the whole recording and can iterate freely.

Online the data arrives one sample at a time and the clock never stops.

Offline accuracy is an upper bound, not a promise – online performance is almost always lower.

Offline analysis

you already have the data



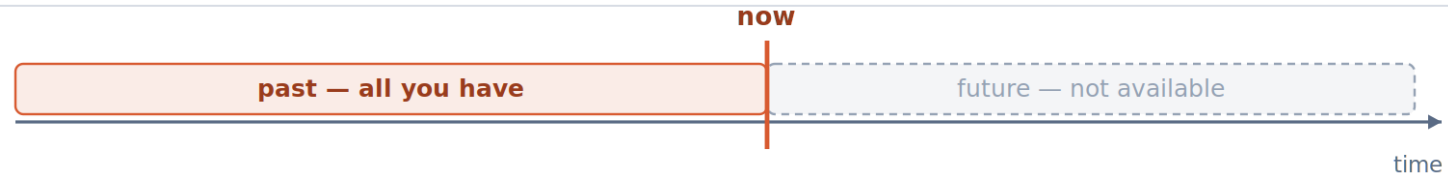
Can look ahead

Unlimited compute time

Re-run until it works

Online analysis

the data arrives one sample at a time



Cannot look ahead

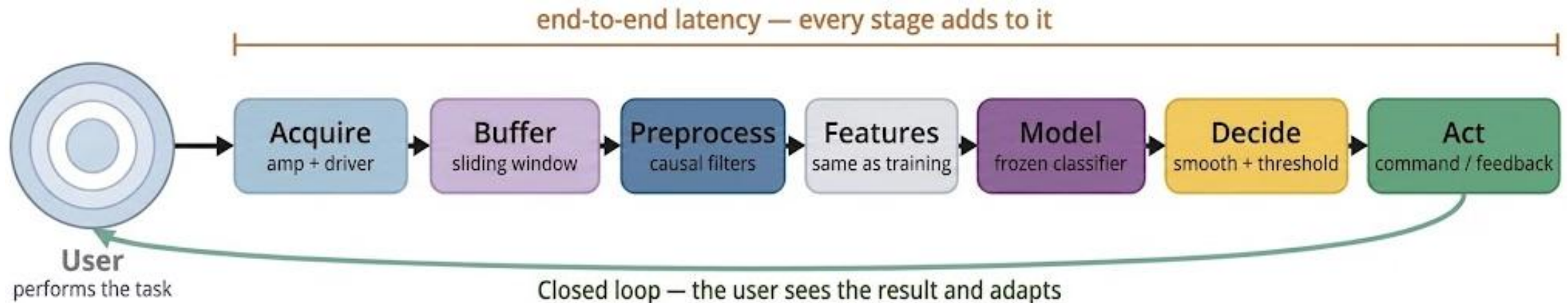
A hard deadline every window

One pass, no do-overs

The real-time loop

Every stage runs continuously on the newest slice of data, and the person is inside the loop – reacting to what the system just did.

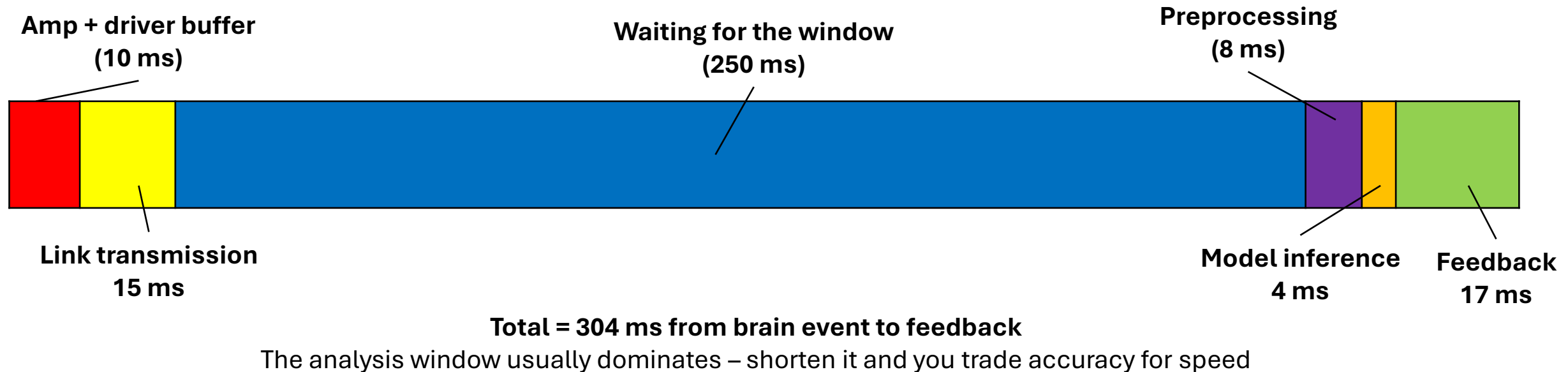
Design the **whole loop**, not the classifier alone as the user is a **component** of the system.



The latency budget

Delay is **cumulative**; every buffer, link and window adds milliseconds between a brain event and the feedback the user sees.

Where the delay actually comes from



Acquisition and streaming

The first job is to get samples off the amplifier and into your code reliably, with trustworthy timestamps and no gaps.

Get the data flowing

Vendor SDK or an open driver reads the amp.

Publish it as a named stream other tools can read.

LSL is the common choice.

Streams, not files

Continuous samples with timestamps.

Task events, separately labelled.

LSL aligns clocks across devices and PCs.

Watch for

Dropped samples and reconnections.

Large device buffers hiding real delay.

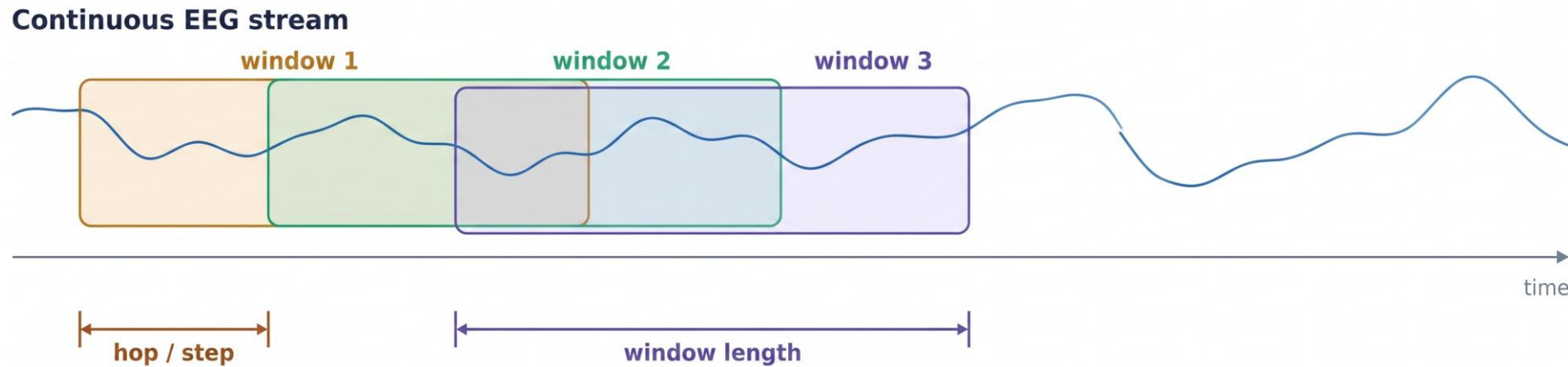
Wireless links adding tens of milliseconds.

Buffering and windowing: Precision vs. speed

The 2.0s window (Precision): Accurately calculating frequencies (Alpha/Beta) requires observing multiple full wave cycles. A 2.0-second block guarantees high mathematical fidelity.

The 0.25s slide (Speed): Waiting 2 seconds for completely fresh data causes severe interface lag. Instead, the window slides forward by just 0.25 seconds, reusing 1.75s of historical data to generate 4 instant predictions per second.

The memory buffer (Efficiency): To prevent RAM overflow during continuous live streaming, a memory loop seamlessly drops the oldest 0.25s of data exactly as the newest 0.25s enters.



Smaller hop → smoother control but more to compute -- Longer window → better estimate but slower

Artifacts in real time

You cannot inspect the recording and clean it by hand.
Artifact handling must be **automatic, fast, and fail-safe**.

Detect

- Range of amplitudes per channel.
- Impedance or contact-quality flags from the amplifier.

Suppress

- Regression or fixed ICA weights learned at calibration.
- Artifact subspace methods trained on clean baseline.

Or simply refuse

- Mark the window bad and skip the decision.
- **Tell the user:** “Signal unclear, try again”.

Live preprocessing

Real-time systems can only process past/present data. "**Look-ahead**" **offline filtering will crash** live streams.

One-way filtering

- Use filters with **forward-only** calculations.
- Expect slight, natural timing delays.

Handling data in pieces

- Pass filter "**memory**" between data chunks to avoid signal glitches (edge artifacts).
- Allow a brief **warm-up period** (it takes time for the filter to adjust).

Scaling

- Scaling logic must **strictly match** offline baseline rules.

IIR vs. FIR filters

Filter	IIR (Infinite Impulse Response)	FIR (Finite Impulse Response)
Primary use	Online: Real-time BCI streaming.	Offline: Post-session analysis.
Speed and latency	Virtually zero latency. Recursive feedback requires minimal buffering to achieve sharp cutoffs.	High computational lag. Requires massive historical data buffering before outputting a signal.
Phase and morphology	Non-linear phase. Delays frequencies unevenly, which distorts and smears the physical shape of the brainwave.	Linear phase. Delays all frequencies equally, perfectly preserving the exact shape and timing of the spike.
BCI application	Ideal for frequency power (e.g., Alpha/Beta rhythms) where total window energy matters, not exact wave shape.	Mandatory for time-domain features (e.g., P300 ERPs) where the exact millisecond of a spike is critical.

Online BCI filter selection: The "Big three" IIR options

Filter type	Primary characteristic	The mathematical trade-off	Best BCI application
Butterworth	Maximally flat passband (zero ripple).	Moderate roll-off steepness.	Frequency-based tasks: Alpha/Beta band power, focus tracking, and MI.
Bessel	Preserves exact wave shape in time (flat group delay).	Very gentle, sluggish roll-off.	Time-based tasks: Peak detection, P300 spellers, and ERPs.
Chebyshev Type II	Brutally steep roll-off while keeping passband flat.	Introduces ripple (instability) into the stopband.	Surgical noise removal: Tight 50 Hz notch filtering without harming adjacent high-Gamma waves.

Features and the frozen model

The single most common cause of “**it worked offline**” failures: The live pipeline **does not compute exactly** what training computed.

Mirror the training path

- Same channels, order, units and sampling rate.
- Same band definitions and window length.
- Same features processing code – ideally the same function.

Never re-fit live

- Do not recompute the parameters on incoming data.
- Do not re-select features per session.
- Fitting only happens at training.

From raw scores to reliable commands

A single prediction from your model is too **unstable** to safely control a system.

The 4-step decision logic includes:

Smoothing

Average the raw, erratic **predictions over time** to create a stable curve (Never issue a command from a single data window).



Decision threshold

Set a specific confidence target (e.g., 80%) that the smoothed score must exceed.



Dwell time

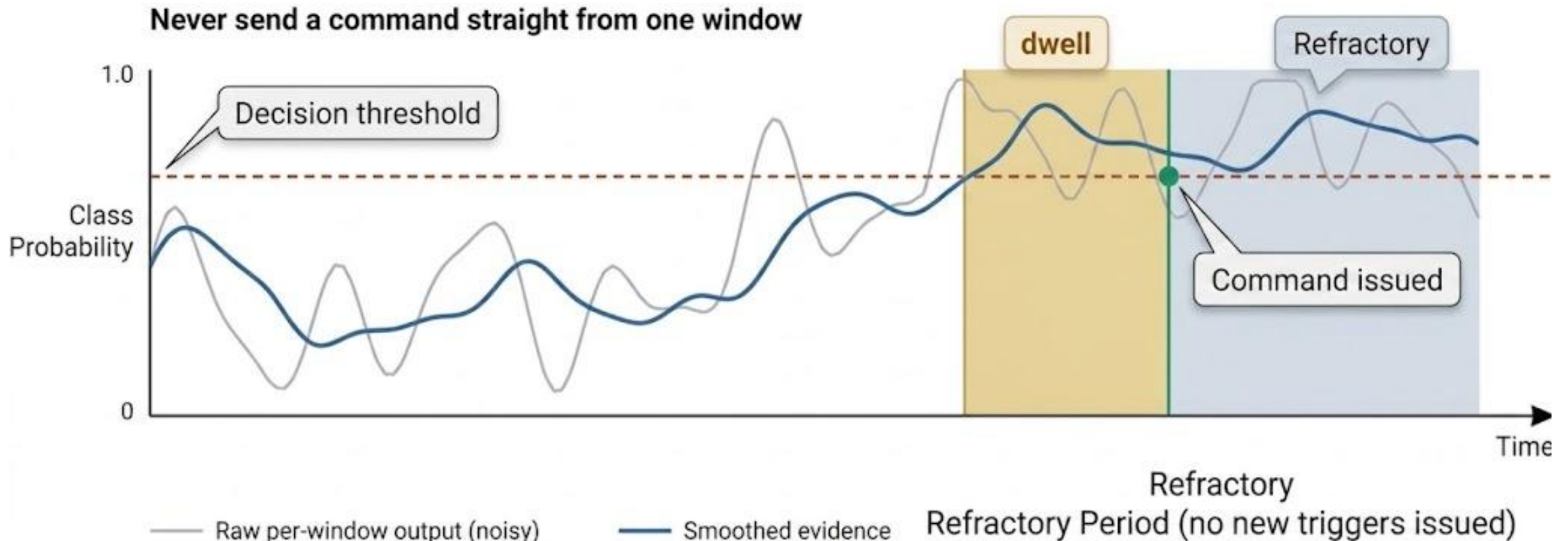
Ensure the score stays above the threshold for a set time (e.g., 1.5 seconds) to confirm it wasn't a random spike.



Refractory period

After issuing the command, enforce a short "cooldown" period where new triggers are ignored to prevent system overload.

From raw scores to reliable commands



Feedback and control output

What the user sees **closes the loop** and shapes how their brain behaves on the very next window.

Make it immediate

- Feedback **must feel** caused by the user's effort.
- Long or variable delay **breaks the sense** of control.
- Keep the update rate **smooth and predictable**.

Make it interpretable

- Show **continuous progress**, not just the final answer.
- Signal confidence and “no decision” states clearly.
- Display signal quality so problems are visible.

Mind the side effects

- **Feedback itself changes the EEG** you are decoding.
- Frustration and fatigue degrade performance fast.
- Reward effort and progress, not perfection.

The calibration session

Almost every live session begins by collecting a short-labelled block from this person, **today**, with this cap on.

A subject-specific calibration is still the **most reliable way** to get good online performance, pretrained models reduce it but rarely remove it.

Set up and check signals

- Fit the cap, check impedance and noise before recording anything.

Run cued trials (if any)

- *A short block with known labels – typically a few minutes.*

Fit the pipeline

- Train filters, feature transforms and the classifier on that block.

Validate quickly

- Cross-validate; if accuracy is at chance, fix it before going live.

Freeze and launch

- Lock the whole bundle, then switch to live decoding.

Adaptation and recalibration

EEG drifts within a session and changes between days. A model that is frozen forever will **slowly go stale**.

Monitor a quality metric continuously; recalibrate **when it degrades**, not on a fixed schedule.

Why it drifts

Fatigue, attention and posture change.

New cap placement on a different day.

What you can adapt

Update the classifier bias and decision boundary.

Retrain on a fresh calibration block.

Timing and synchronization in evoked-based tasks

The hardware truth

The exact millisecond a stimulus appears (e.g., a screen flash), a hardware trigger **should** mark the EEG stream. This is your "True event."

Software lag (Offset)

Software markers sent over a network or OS often **arrive late**.

If this delay is **the same** every time (a Fixed offset), it is **easy to fix**; you simply measure the delay and shift your data backward.

The threat of jitter

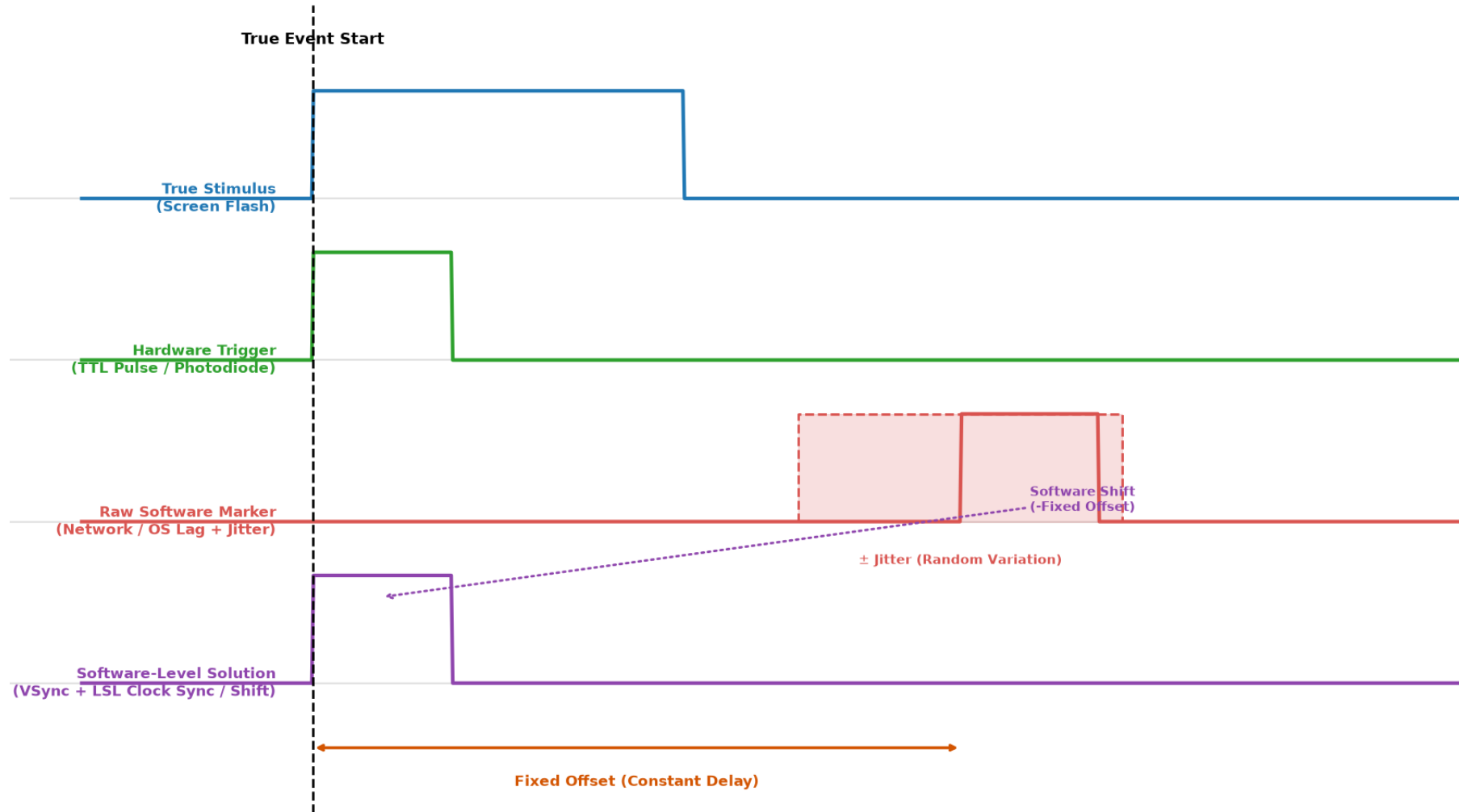
If the software delay **varies randomly** from trial to trial (e.g., sometimes late by 10ms, sometimes by 80ms), this is Jitter.

When you average **jittery epochs** together, the brain waves **misalign and smear** the signal into a flat line. Jitter cannot be fixed after the fact.

The solution

Use **LSL clock synchronization** to continuously measure and correct for network drift.

Timing and synchronization in evoked-based tasks



Electrical safety and grounding

You are attaching conductive electrodes to a person's head. Electrical safety is **non-negotiable** and comes before any research goal.

Isolation

- Prefer battery power or certified isolated supplies.
- Never wire a participant to a non-isolated device.

Standards and checks

- Inspect cables and connectors before every session.
- Retire damaged leads immediately – **do not repair**.

Grounding and noise

- Keep the participant away from main cabling.
- The EEG ground is a reference, not a safety earth.

The recording environment

Few minutes spent on setup routinely beats hours of algorithmic rescue work.

Electrical noise

- Distance from mains cables, chargers, motors and fluorescent lights.

Screen and lighting

- Stable refresh for visual paradigms; avoid flicker interference.

Participant comfort

- Supported posture reduces muscle artifact more than any filter.

Movement control

- Fix cables, brief the participant, minimize jaw and neck tension.

Session hygiene

- Consistent cap placement, impedance log, and standard checklist.

Distraction control

- Quiet room, phones away, and no interruptions mid-run.

Matching the system to the task

Different paradigms stress different parts of the system. The pipeline you build should follow from the paradigm you chose.

Paradigm	Critical need	Hardware priority
P300 speller	Exact stimulus timing	Reliable triggers
SSVEP	Stable flicker rendering	Fast display and occipital coverage
Motor imagery	Continuous decoding	Good sensorimotor coverage
Neurofeedback	Fast and smooth updates	Low latency and few channels suffice
Passive monitoring	Long-session stability	Comfort seating and dry electrodes

Measuring online performance

For any system that can act on the world, a false activation is usually **worse** than a missed one, however, report both.

Task-level

Command accuracy

ITR

Task completion rate for the real application

Commands issued when the user did nothing

System-level

End-to-end latency

Signal quality and artifact rejection rate

Does performance hold across the session?

Workload, fatigue, and sense of control

Failure modes and fallbacks

Every live system needs a big, obvious **stop control** that does not depend on the pipeline working.

Electrode comes loose

- Detect via **quality flags**; pause decoding and prompt a re-check.

Stream drops out

- Reconnect **automatically**; hold the last safe state meanwhile.

Deadline missed

- **Skip** the window rather than queueing stale decisions.

Accuracy collapses

- Fall **back to calibration**; do not let the user keep failing.

Runaway output

- **Hard stop**, plus a manual override the operator always controls.

Participant distress

- Stop **immediately**, comfort outranks data without exception.

Neuroethics: Core principles

Going live means a real person's brain data drives a real system. That raises duties beyond ordinary engineering.

Mental privacy

Neural data is uniquely sensitive.

Collect the minimum; protect it accordingly.

Autonomy and agency

The user must stay in control and able to stop.

Consent must be informed, ongoing and revocable.

Ethics in your own study

Record the participant's **right to withdraw**, and make sure withdrawing deletes their data.

Get approval first

- Ethics committee review before any human recording.

Consent properly

- Explain the purpose, risks, data use and the right to stop at any moment.

Govern the data

- Pseudonymize, store securely, define retention and who may access it.

Manage expectations

- State clearly what the system does and does not infer.

Protect the vulnerable

- Extra safeguards for minors, patients and anyone in a dependent role.

Go-live readiness checklist

Run through this **before the participant arrives**; every item has cost someone a session.

If any item is unchecked, you are not running a live session, you are running an experiment on your own pipeline.

Pipeline verified

- Replay and pseudo-online tests pass; features match training exactly.

Timing measured

- Marker offset and jitter quantified; latency budget known.

Hardware checked

- Cables intact, isolation confirmed, impedances acceptable.

Logging on

- Raw stream, markers, decisions and model version all recorded.

Fallbacks ready

- Stop control, reconnect logic and operator view tested.

Ethics in order

- Approval, consent forms, data plan and a briefed participant.

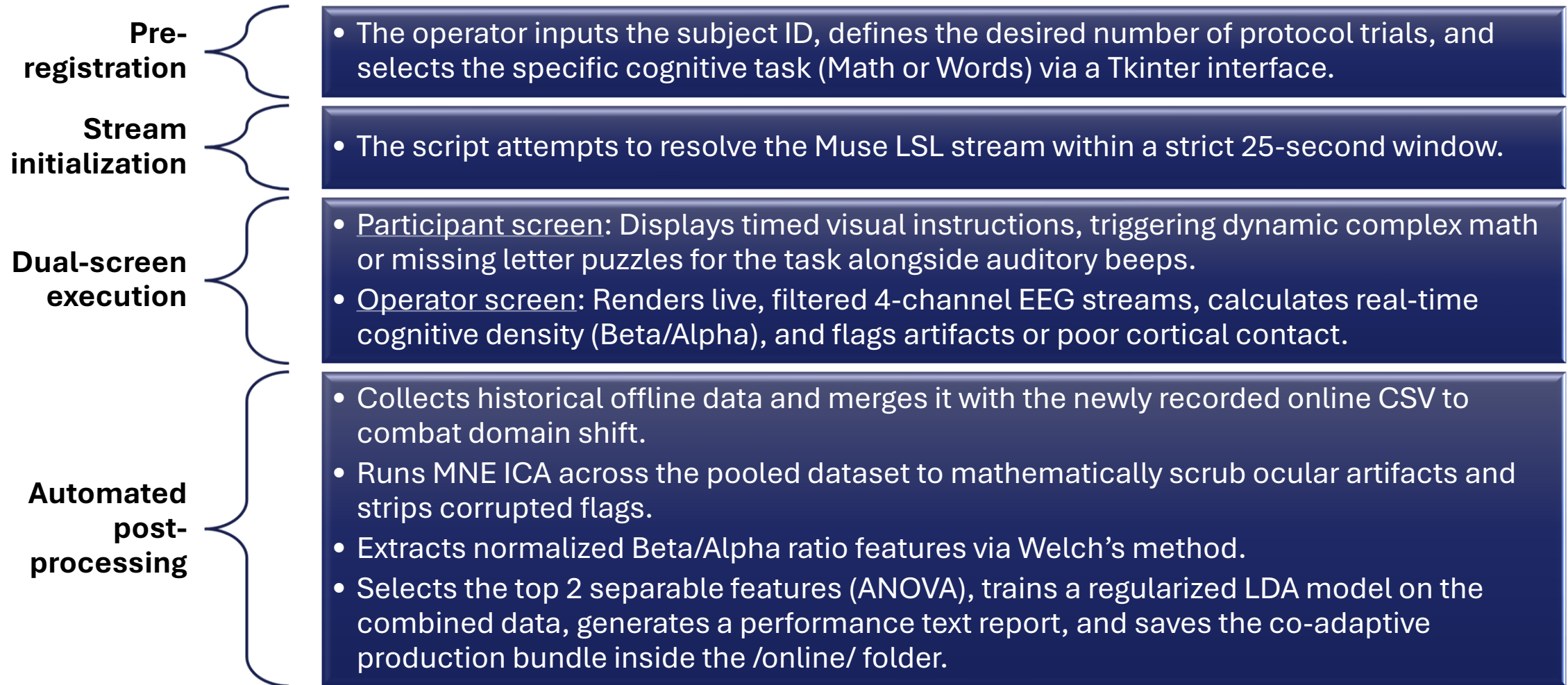
Muse 2 online pipeline

Calibration before going live

Overview of utilized libraries

os, sys, time, threading, multiprocessing, collections (deque)	Manages inter-process communication (IPC), directory handling, synchronized timers, and rolling data buffers.
numpy, pandas, scipy (signal)	Handles fast matrix operations, CSV data structuring, and causal linear filtering.
pyslsl, muselsl, mne	Establishes Bluetooth headset connections, resolves LSL streams, and executes advanced offline ICA artifact scrubbing.
sklearn, joblib	Executes ANOVA feature selection, Regularized LDA training, model evaluation metrics, and object serialization.
pygame, winsound, tkinter	Renders the NOFRAME dual-screen displays, generates auditory protocol triggers, and manages the pre-registration menu.

The script's pipeline



Pipeline initialization and streaming

Subject registration

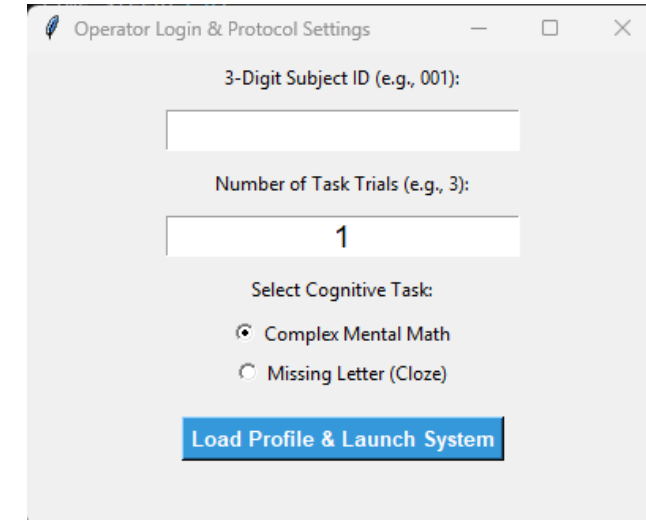
- Operator assigns the 3-digit subject ID, defines the required number of task trials, and explicitly selects the targeted cognitive task via the startup UI.

LSL handshake

- The system initiates an automated 25-second polling window to connect with the Muse headset's LSL.

Multiprocessing architecture

- Launch of simultaneous, isolated processes for data streaming, operator diagnostics, and the participant visual protocol.



Operator Login & Protocol Settings

3-Digit Subject ID (e.g., 001):

Number of Task Trials (e.g., 3):

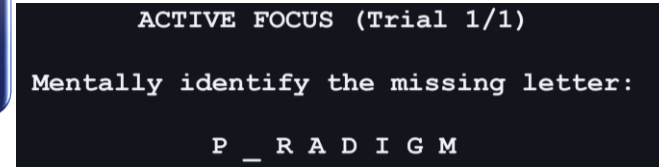
1

Select Cognitive Task:

☒ Complex Mental Math

☐ Missing Letter (Cloze)

Load Profile & Launch System



ACTIVE FOCUS (Trial 1/1)

Mentally identify the missing letter:

P _ R A D I G M

Dual-screen live diagnostics

Causal filtering engine

- Applies **rapid filter** to continuous raw data for zero-latency operator visualization.

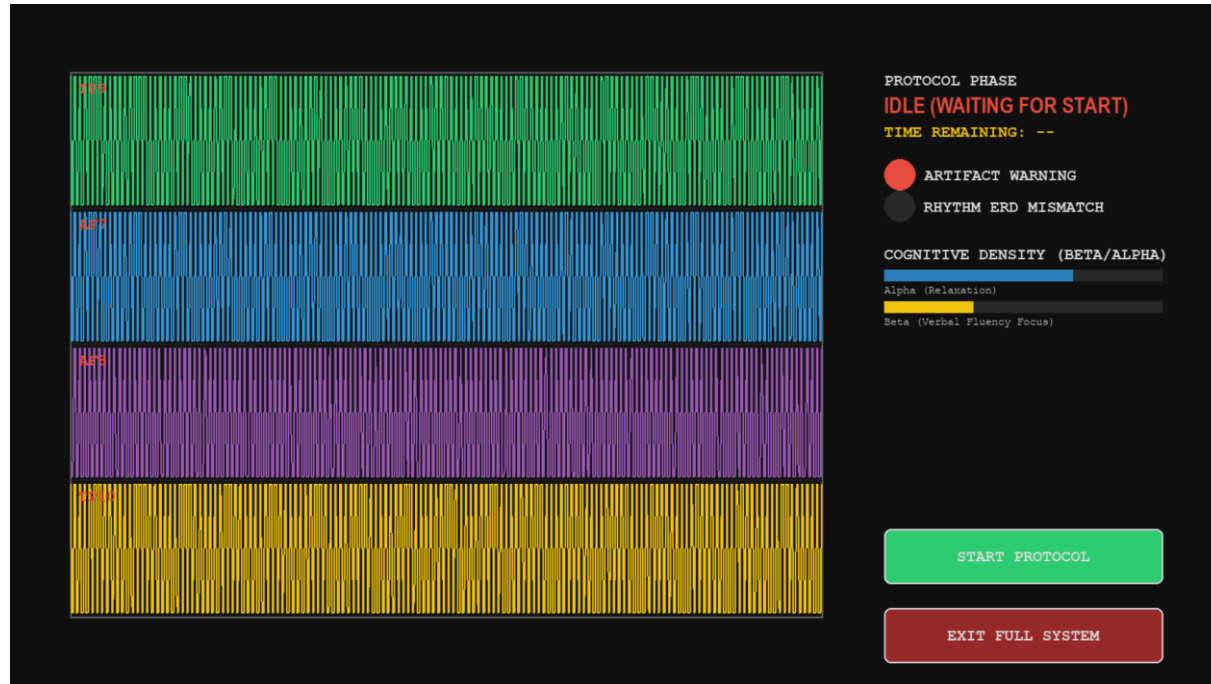
Signal quality assurance

- Automatically monitors peak-to-peak voltage ($>350\mu\text{V}$) and variance ($<2.0\mu\text{V}$) to trigger real-time artifact and flatline warnings.

Cognitive density tracking

- Continuously visualizes the ratio of Alpha (relaxation) to Beta (focus) power to monitor physiological task engagement.

Dual-screen live diagnostics



RESTING PHASE (Trial 1/1)

Keep your eyes open and relax.

Task protocol execution

Dynamic verbal task generation

- Participants view technical terms with a single missing character replaced by an underscore (e.g., S Y S T _ M) and refresh the prompt every two seconds to drive continuous working memory retrieval.

Dynamic math task generation

- Participant is presented with randomized arithmetic problems formatted as $X \pm X \pm X = ?$ to solve mentally without external scratchpads.

Automated state machine

- Protocol automatically cycles through eyes open baseline, eyes closed baseline, resting phase, and active focus states.

Synchronized logging

- Timestamps, raw 4-channel data, paradigm markers, and real-time artifact flags are continuously exported to a local CSV matrix.

Mathematical artifact scrubbing using MNE

Offline transition

- Upon protocol completion, pygame and LSL processes terminate to free system RAM for intensive offline computation.

Spatial unmixing

- The saved CSV is converted to an MNE array to perform ICA.

Ocular extraction

- The **AF7** electrode acts as a **proxy EOG**, allowing the pipeline to mathematically zero out blink components while reconstructing continuous brainwaves.

Feature engineering and extraction (Co-Adaptive)

Target slicing and data pooling

- The system dynamically **merges** the newly recorded live calibration CSV with the historical offline CSVs, slicing the continuous arrays to isolate only Marker 3 (Relax) and Marker 4 (Focus) task data.

Welch's PSD calculation

- Converts the time-domain epochs from the pooled dataset into frequency power densities.

Metric normalization

- Calculates the absolute Alpha (8-12 Hz) and Beta (13-30 Hz) power, then derives the strictly normalized Beta/Alpha ratio for each electrode.

Automated ML training and export (Co-Adaptive)

Pooled ANOVA pruning

- Evaluates the extracted ratios across the combined dataset to isolate the top 2 features demonstrating the highest mathematical separability between Rest and Focus.

Co-adaptive LDA classification

- Trains a regularized LDA model (LSQ solver, automated shrinkage) using the **fused offline and online datasets**, effectively bridging the domain shift between open-loop and closed-loop neural signatures.

Production packaging (serialization)

- Automatically executes 5-fold cross-validation on the combined matrix, generates a rigorous textual performance report, and exports the final co-adaptive system bundle directly to the /online/ directory.

*Thank
You!*



HERBioLab
